Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

5. Implement tokenization and segmentation of text using regular expressions in NLTK

Aim: To split text into sentences, words, and meaningful tokens using regular expressions.

```
import re
import nltk
from nltk.tokenize import RegexpTokenizer, sent_tokenize

# Sample text containing two sentences and punctuation.
text = "NLTK is useful for NLP. Regular expressions tokenize text!"

# Step 1: Sentence segmentation.
sentences = sent_tokenize(text)

print("Sentence Segmentation:")
for sentence in sentences:
    print(sentence)

# Step 2: Word tokenization using NLTK's regular-expression tokenizer.
# The pattern selects sequences of alphabetic characters.
tokenizer = RegexpTokenizer(r'\w+')

words = tokenizer.tokenize(text)

print("\nWord Tokenization:")
print(words)

# Step 3: Extract numbers using a regular expression.
numbers = re.findall(r'\d+', "There are 25 students in 2 labs.")

print("\nNumbers found:")
print(numbers)
```

Sample output

```
Sentence Segmentation:
NLTK is useful for NLP.
Regular expressions tokenize text!

Word Tokenization:
['NLTK', 'is', 'useful', 'for', 'NLP', 'Regular',
 'expressions', 'tokenize', 'text']

Numbers found:
['25', '2']
```

Explanation:

• `sent_tokenize()` divides text into sentences.

• `RegexpTokenizer(r'\w+')` extracts word-like tokens and excludes punctuation.

- `re.findall(r'\d+', text)` extracts sequences of digits.